Predicting Routing Congestion from Chip Placements with a Lightweight Convolutional Encoder–Decoder

Faria Samanta Akbar Student #: 1010708183 faria.akbar@mail.utoronto.ca

Code: https://github.com/FariaSamanta/APS360-Chip-Congestion
Colab: https://colab.research.google.com/drive/1TjeERbnvFhNeIo_u3M3cmsqiBg2814pv

1 Introduction

Before a chip is manufactured, millions of standard cells are *placed* on the die and then wires are *routed* between them. If too many wires must pass through one region, the layout is *congested*: routing fails or produces design-rule violations, and engineers must re-place and re-route, a loop in which each iteration of detailed routing can take hours to days. Being able to see congestion *immediately after placement* lets engineers and automated optimizers explore far more layouts per day. This is why EDA vendors now ship AI-driven design optimization and why start-ups targeting AI-for-chip-design are raising large rounds [1].

Goal. Given image-like feature maps extracted from a placed (unrouted) layout, predict two per-tile maps of *global-routing overflow* (horizontal and vertical) in milliseconds, and generalize to *designs never seen during training*.

Why deep learning. The inputs and outputs are spatially aligned 2-D maps, and congestion arises from local interactions (macro blockages, pin clusters, wire demand) at several scales. This is exactly the structure that convolutional encoder–decoders capture well, while hand-crafted estimates such as RUDY [2] only measure local wire density and miss how demand spreads around obstacles.

2 Illustration

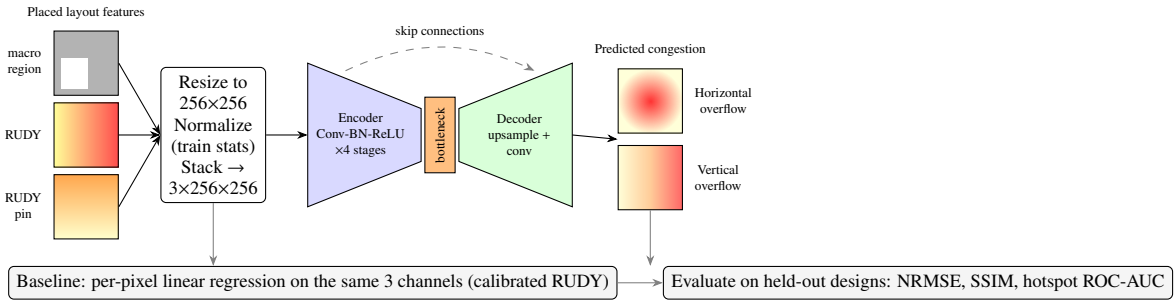


Figure 1: Proposed pipeline. Three placement-stage feature maps are preprocessed and passed through a lightweight U-Net-style encoder–decoder that outputs horizontal and vertical routing-overflow maps. A non-neural baseline is trained on the same inputs, and both are scored on designs held out from training.

3 Background & Related Work

RUDY [2] estimates routing demand by spreading each net's expected wirelength uniformly over its bounding box. It is fast and still used inside commercial placers, but it ignores blockages and detours. It serves as both an input feature and the basis of our baseline. **RouteNet** [3] was among the first to treat routability as an image problem, using a fully convolutional network to predict DRC hotspots for mixed-size designs. **Liu et al.** [4] trained an FCN to predict congestion inside a global placer and back-propagated through it to optimize routability. Their model is the official congestion reference in CircuitNet (NRMSE 0.040, SSIM 0.80). **MAVIREC** [5] applied a U-Net [6] to IR-drop estimation, showing that encoder–decoders with skip connections suit dense physical-design maps. **CircuitNet** [7, 8] released over 10k real layouts with aligned feature and label maps, making reproducible benchmarking possible. **Mirhoseini et al.** [9] used reinforcement learning for macro placement and depended on a fast proxy for congestion, which illustrates why cheap, accurate predictors are a bottleneck for learned placement. **This project** differs by (i) using a lightweight model and reporting accuracy against parameter count, (ii) evaluating strictly on *unseen designs*, and (iii) quantifying how much a random split overstates accuracy.

4 Data Processing

Source. CircuitNet-N28 [7]: 10,242 successful layouts of six RISC-V designs (RISCY-a/b, RISCY-FPU-a/b, zero-riscy-a/b) in a 28 nm process, each produced with different placement and routing settings. Feature maps are tile-based (2.25 μm tiles, about 300×300 per layout). **Inputs:** macro_region, RUDY, RUDY_pin. **Labels:** global-routing horizontal and vertical overflow. **Steps.** (1) Download only the four needed archives (4.6 GB compressed, 28 GB once unpacked, versus about 200 GB for the full release) and decompress them; they are gzipped twice. (2) Parse the design name from each file name (e.g. 1-RISCY-a-1-c2-u0.7-m1-p1-f0) and check that all five maps exist for each sample (verified: 10,242/10,242). (3) Resize to 256×256: nearest-neighbour for the binary macro map, bilinear for the continuous maps. (4) Clip each channel at its 99.9th percentile and scale to $[0, 1]$, using statistics computed *on training designs only*. (5) **Split by design**: train = RISCY-a, RISCY-b, RISCY-FPU-a (5,830 layouts, 57%); validation = RISCY-FPU-b (1,248, 12%); test = zero-riscy-a/b (3,164, 31%), an unseen core. A random split is also run purely to measure leakage. (6) Augment with flips and 90° rotations; a rotation swaps the horizontal and vertical label channels, so we swap them accordingly. All steps are scripted and saved as compressed .npz files with the normalization statistics, so they can be reproduced exactly.

5 Architecture

A lightweight U-Net-style encoder–decoder [6] written in PyTorch. Encoder: 4 stages, each two 3×3 Conv–BatchNorm–ReLU layers followed by 2×2 max-pooling, with widths 16–32–64–128. Bottleneck: dilated 3×3 convolutions to enlarge the receptive field, since congestion propagates over long distances. Decoder: bilinear upsampling, concatenation with the encoder skip feature, and two Conv–BN–ReLU layers per stage. Head: a 1×1 convolution to 2 channels followed by a sigmoid. Loss: $L = \text{MSE} + \lambda(1 - \text{SSIM})$ [10]. Optimizer: Adam (initial learning rate 10^{-3}, cosine decay), batch size 16, with early stopping on validation NRMSE. Ablations: removing skip connections, removing dilation, model width, and MSE-only loss.

6 Baseline Model

Calibrated RUDY: a per-pixel linear regression $\hat{y}_c(p) = \mathbf{w}_c^\top[\text{macro}(p), \text{RUDY}(p), \text{RUDY_pin}(p), 1]$, fit by least squares on the training designs separately for each output channel c. It is trivial to reproduce and represents what an engineer gets from the standard heuristic. We also report raw RUDY thresholding for hotspot detection.

7 Evaluation & Success Criteria

Metrics follow CircuitNet: NRMSE and SSIM on the overflow maps, plus ROC-AUC for *hotspot* pixels (label above the 90th training percentile), on the held-out designs. Success means (a) clearly beating the calibrated-RUDY baseline on all three metrics, (b) approaching the reference NRMSE/SSIM with substantially fewer parameters, and (c) reporting inference time per layout and the gap between the random and design-held-out splits.

8 Ethical Considerations

The data contains no personal information; it comes from open-source RISC-V cores released for research. The main risk is *over-reliance*: a false "clean" prediction could lead engineers to skip checks and cause a costly re-spin, so the model should be used as an advisory, early-stage tool and never as sign-off. Because the data covers one 28 nm node and six designs from three cores, accuracy will likely degrade on other nodes and design styles; we will state this limitation explicitly. If such models are trained on proprietary layouts, they could leak confidential design IP, and wider automation may change physical-design jobs. Both are worth noting even though this project uses only open data.

9 Project Plan & Risks

Individual project. **Done (Oct 15):** data downloaded and integrity-checked. **Oct 16–30:** preprocessing, EDA, baseline. **Nov 1–19:** first U-Net results (progress report, Nov 20). **Nov 21–Dec 4:** tuning, ablations, held-out evaluation. **Dec 5–8:** final report. **Risks:** Colab disk is wiped between sessions, so the preprocessed 256×256 arrays (about 7 GB in float16) are saved once and reloaded; if memory is tight, we train on a subset (about 600 layouts per design); if Colab GPU time runs short, we reduce the input to 128×128.

References

[1] TAM Radar. Cognichip raises $60M series A for AI chip design. https://www.tamradar.com/funding-rounds/cognichip-series-a-60m, 2026. Accessed Oct. 15, 2026.

[2] Peter Spindler and Frank M. Johannes. Fast and accurate routing demand estimation for efficient routability-driven placement. In *Design, Automation & Test in Europe Conference (DATE)*, 2007.

[3] Zhiyao Xie, Yu-Hung Huang, Guan-Qi Fang, Haoxing Ren, Shao-Yun Fang, Yiran Chen, and Jiang Hu. RouteNet: Routability prediction for mixed-size designs using convolutional neural network. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.

[4] Siting Liu, Qi Sun, Peiyu Liao, Yibo Lin, and Bei Yu. Global placement with deep learning-enabled explicit routability optimization. In *Design, Automation & Test in Europe Conference (DATE)*, 2021.

[5] Vidya A. Chhabria, Yanqing Zhang, Haoxing Ren, Ben Keller, Brucek Khailany, and Sachin S. Sapatnekar. MAVIREC: ML-aided vectored IR-drop estimation and classification. In *Design, Automation & Test in Europe Conference (DATE)*, 2021.

[6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[7] Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Wei Liu, Runsheng Wang, and Ru Huang. CircuitNet: An open-source dataset for machine learning applications in electronic design automation (EDA). *Science China Information Sciences*, 2022. doi: 10.1007/s11432-022-3571-8.

[8] Xun Jiang, Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Runsheng Wang, and Ru Huang. CircuitNet 2.0: An advanced dataset for promoting machine learning innovations in realistic chip design environment. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, et al. A graph placement methodology for fast chip design. *Nature*, 594:207–212, 2021.

[10] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.